

CAIRN for AI Pentesting Agents

Find repeated tool-output work, stale replay risk, protected state changes, and safe context savings without sending raw logs outside the customer environment.

The problem

AI-pentest agents repeatedly inspect scanner output, shell output, exploit framework state, HTTP responses, search results, target files, and repeated failed attempts.

The hard part is safety. In pentesting workflows, naive caching is dangerous. Target and session state can change after a login, exploit attempt, shell command, file write, service restart, or new scan result.

CAIRN is not a vulnerability scanner. It is not claiming flaws in Aikido or any security product. It audits trace efficiency and stale replay risk.

What CAIRN tests

- Where is the agent repeating expensive tool-output/context work?
- Where would exact replay be stale or unsafe?
- Where is exact cache actually safe?
- Where is live delta-serving the better option?
- What would token and dollar savings have been?

Protected audit lane

repeated observation found
-> check protected target/session state
-> changed state: block stale exact replay
-> safe cell: exact cache or live delta-serve
-> report savings with receipts

Public security-agent receipt

AutoPenBench / genai-pentest-paper public experiment logs. Benchmark receipt, not production Aikido traffic.

2,764 tool events audited	1,031 re-reads found
37.30% repeated work	548K point tokens avoided
3.70M carried-context tokens avoided	81.08% avoided-token ratio on re-read traffic
1,016 protected-lane blocks	0 stale serves / false hits

Repeated families found in the public pentest-agent run

- nmap
- curl
- ssh
- msfconsole
- searchsploit
- find
- cat
- python3
- search

What the report shows

- Repeated-work percentage, re-reads, top tool families, and concrete examples.
- Exact-cache stale-risk events and protected target/session-state blocks.

- DELTA_SERVE, EXACT_CACHE, live-only cases, and estimated token/dollar savings using user-supplied model prices.
- Output artifacts: summary.json, summary.md, and report.html / PDF.

Design-partner pilot

1**Run locally**

Use a day or week of AI-pentest traces. Raw logs stay inside the customer environment.

2**Review report**

Share only the generated report, or review the result over screenshare.

3**Decide**

If there is signal, pick one high-volume repeated tool family for a protected pilot.

The ask

Run CAIRN locally on a day or week of Aikido AI-pentest traces. If there is no repeated-work signal, stop. If there is signal, evaluate one protected reuse pilot.

CAIRN = agent trace/context efficiency. fraQtl = model/runtime efficiency.

Audit-only v0: no production integration, no auto-serving, no pentest actions executed.